

Sample Final – Introduction to Python

6013B0470Y Introduction to Python 24/25 (2.2) · 1 exercise · 24.0 points

1 Question 1

24.0 points · 24 questions

a You'd like to define a class called `Vehicle`. It should have two attributes: `name` and `mode`, which you want to be initialized during object creation.

The `name` attribute can be any string, and it must be passed to the object during creation. The `mode` attribute is also a string, but it doesn't necessarily need to be passed to the object constructor. If it isn't specified at object construction, then the `mode` attribute should be equal to the string `"land"`.

For example, if we create a `Vehicle` object as:

```
my_car = Vehicle("Mazda")
```

then the following two conditions should both be true:

```
my_car.name == "Mazda"
```

```
my_car.mode == "land"
```

Which of the following code segments achieves what you want?

1.0 point · Multiple choice · 4 alternatives

- ☒

```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode="land"):
        self.name = vehicle_name
        self.mode = vehicle_mode
```
- ☐

```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode):
        self.name = vehicle_name
        self.mode = vehicle_mode
```
- ☐

```
class Vehicle:
    def __init__(self, vehicle_name, vehicle_mode="land"):
        return name, mode
```
- ☐

```
class Vehicle:
    def __init__(vehicle_name, vehicle_mode):
        name = vehicle_name
        mode = vehicle_mode
```

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

b Take a look at the following class definition.

```
class Book:
    def __init__(self, title, author, review_scores=None):
        self.title = title
        self.author = author
        self.review_scores = review_scores if review_scores else []

    def add_review(self, score):
        self.review_scores.append(score)

    def show_rating(self):
        if self.review_scores:
            return round(sum(self.review_scores) /
len(self.review_scores), 1)
```

Suppose that we create a Book object and call its add_review method three times, like this:

```
book_1 = Book("The Lightning Thief", "Rick Riordan")
book_1.add_review(5)
book_1.add_review(3)
book_1.add_review(3)
```

What will then the following command print to the screen?

```
print(book_1.show_rating())
```

1.0 point · Multiple choice · 4 alternatives

☒ 3.7

☐ 4

☐ None

☐ 3.67

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

c Imagine that you have a list called `students`, which is populated by a number of dictionaries.

Each dictionary in the `students` list has two keys: `"Name"` and `"Grade"`. The corresponding values are student names (as strings) and course grades (as floats).

For each student, you'd like to print the following text to the screen:

`[NAME]` has received a grade of `[GRADE]`.

where `[NAME]` is substituted by the name of the student and `[GRADE]` is substituted by the actual course grade. The course grade should be displayed to one decimal place after the decimal point.

For example, if

```
students = [{"Name": "Adam", "Grade": 75}, {"Name": "Bernard", "Grade": 80}]
```

then you'd like to see the following two lines printed to the screen:

Adam has received a grade of 75.

Bernard has received a grade of 80.

Which of the following code segments will achieve what you want?

1.0 point · Multiple choice · 4 alternatives

- ☒

```
for student in students:  
    print(f"{student['Name']} has received a grade of {student['Grade']}")
```
- ☐

```
for student, grade in students.items():  
    print(f"{student} has received a grade of {student['Grade']}")
```
- ☐

```
for student in students:  
    print(f"{student['Name']} has received a grade of " +  
          str(student["Grade"]))
```
- ☐

```
for student, grade in students.items():  
    print(f'{student} has received a grade of {grade}.')
```

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

d You need to write a function called `format_phone_number` that formats phone numbers. The output of the function should be a string that represents a 10-digit number. The input argument is also a string with 10 digits, but the digits may be separated by dashes (the "-" character) at various places.

For example, the following function calls:

```
format_phone_number("020-525-1400")
```

```
format_phone_number("020-5251400")
```

```
format_phone_number("020-525-14-00")
```

should all return the string:

```
"0205251400"
```

Which of the following code segments **does not** achieve what you want?

1.0 point · Multiple choice · 4 alternatives

- ☒

```
def format_phone_number(number):  
    number.replace("-", "")  
    return number
```
- ☐

```
def format_phone_number(number):  
    return number.replace("-", "")
```
- ☐

```
def format_phone_number(number):  
    digits = []  
    for char in number:  
        if char in "0123456789":  
            digits.append(char)  
    return "".join(digits)
```
- ☐

```
def format_phone_number(number):  
    return "".join(number.split("-"))
```

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

e You have a pandas DataFrame called `df`. It has a column called "A" and a column called "B". Both contain numbers.

You'd like to create a column called "C", which contains the sum of the numbers in columns "A" and "B". Which of the following code lines will achieve what you want?

1.0 point · Multiple choice · 4 alternatives

☒ `df.loc[:, "C"] = df.loc[:, "A"] + df.loc[:, "B"]`

☐ `df.loc["C"] = df.loc["A"] + df.loc["B"]`

☐ `df.loc["C"] = df.loc[:, ("A" + "B")]`

☐ `df.loc["C"] = df.loc["A" + "B"]`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

f You have a pandas DataFrame called df that looks like this, when printed:

	A	B	C	D
1	9.1	9.5	9.0	6.0
2	6.4	8.2	7.5	1.7
3	2.5	5.3	8.8	3.0
4	6.7	3.6	7.2	6.9
5	8.4	1.9	8.2	3.3
6	8.1	8.4	9.9	2.2

Suppose that you'd like to create a new DataFrame that looks like this, when printed:

	B
2	8.2
4	3.6
6	8.4

Which of the following code lines will achieve what you want?

1.0 point · Multiple choice · 4 alternatives

☒ `df.loc[df.index % 2 == 0, ["B"]]`

☐ `df[2, 4, 6, "B"]`

☐ `df.iloc[[2, 4, 6], 2]`

☐ `df.loc[2::2, "B"]`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

g There are two pandas DataFrames: df1 and df2. They look like this, when printed:

```
print(df1)
```

	A	B	C	D
1	5.0	8.2	9.5	4.4
2	6.0	2.4	9.1	3.3
3	3.2	5.2	8.9	3.6
4	7.8	8.7	7.9	8.0
5	1.2	9.7	2.9	3.0

```
print(df2)
```

	A	C	D
4	7.8	7.9	8.0
3	3.2	8.9	3.6
2	6.0	9.1	3.3

Which of the following code lines **could not** have created df2 from df1?

1.0 point · Multiple choice · 4 alternatives

- ☒ `df2 = df1.sort_index(ascending=False).loc[:,["A", "C", "D"]]`
- ☐ `df2 = df1.iloc[[3, 2, 1], [0, 2, 3]]`
- ☐ `df2 = df1.loc[2:4, ["A", "C", "D"]].sort_values("D", ascending=False)`
- ☐ `df2 = df1.loc[2:4, ["A", "C", "D"]].sort_values("C")`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

h There is a pandas Series called `s`, which contains Dutch municipality and corresponding province names, separated by a semicolon and a whitespace. The following, for example, is an element of this Series: "Amsterdam; Noord-Holland".

You'd like to create a new Series that only contains the municipality names from `s` in the same order. Which of the following code lines will achieve what you want?

1.0 point · Multiple choice · 4 alternatives

☒ `s.map(lambda x: x.split("; ")[0])`

☐ `s.split("; ")[0]`

☐ `s.lambda(x.split("; "))`

☐ `s.str[:9]`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

i Take a look at the following code segment.

```
from datetime import datetime

def get_months(date_strings):
    dates = []
    for date_str in date_strings:
        if "-" in date_str:
            dates.append(datetime.strptime(date_str, "%d-%m-%Y"))
        elif "/" in date_str:
            dates.append(datetime.strptime(date_str, "%m/%d/%Y"))
        else:
            dates.append(None)
    return [date.month if date is not None else None for date in dates]
```

Suppose that you call the function like this:

```
get_months(["03/02/2013", "03.02.2013", "03-02-2013"])
```

What does the function call return?

1.0 point · Multiple choice · 4 alternatives

☒ [3, None, 2]

☐ [2, None, 3]

☐ [3, 2]

☐ [2, 3]

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

j Suppose that you want to write a function called `day_of_year` that takes a `datetime` value and returns the number of the day that the `datetime` value represents within the year. The return value is therefore always an integer between 1 and 366.

For example, the function call:

```
day_of_year(datetime(2022, 2, 1))
```

should return the integer:

32

Which of the following code segments will achieve what you want? (You can assume that the `datetime` and `timedelta` classes are already imported from the `datetime` module.)

1.0 point · Multiple choice · 4 alternatives

- ☒

```
def day_of_year(dt):  
    return (dt - datetime(dt.year, 1, 1)).days + 1
```
- ☐

```
def day_of_year(dt):  
    return timedelta(dt - datetime(dt.year, 1, 1)) + 1
```
- ☐

```
def day_of_year(dt):  
    return dt - datetime(dt.year, 1, 1) + 1
```
- ☐

```
def day_of_year(dt):  
    return (dt - datetime(dt.year, 1, 0)).days
```

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

k You have a list of strings called `list_1`, in which every element is unique.
Which of the following code lines will create a dictionary, in which the keys are the elements of `list_1` and the values are the number of characters in the corresponding key?

1.0 point · Multiple choice · 4 alternatives

- ☒ `{item: len(item) for item in list_1}`
- ☐ `[len(item) for item in list_1]`
- ☐ `{item for item in list_1 if len(item) > 0}`
- ☐ `{len(item): item for item in list_1}`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

l Suppose that you have a list of letters called `list_1`. Some letters are upper case, others are in lower case.

You'd like to create another list that contains the same letters in the same order, but you want to turn every upper case letter into its lower case version, and every lower case letter into its upper case version.

Which of the following code lines will achieve what you want?

1.0 point · Multiple choice · 4 alternatives

- ☒ `[letter.upper() if letter.islower() else letter.lower() for letter in list_1]`
- ☐ `[x if x in 'abcdefghijklmnopqrstuvwxyz'.upper() else x.lower() for x in list_1]`
- ☐ **Both** achieve what you want
- ☐ **None** achieves what you want

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

m Suppose you have three variables x, y, z, as follows:

```
x = '3'
```

```
y = 3
```

```
z = 3.0
```

What are the outputs of the following lines of code?

```
print(x == y)
```

```
print(y == z)
```

```
print(x == z)
```

```
print(x == y == z)
```

1.0 point · Multiple choice · 4 alternatives

False

☒ True
False
False

True

☐ True
True
True

False

☐ False
False
True

True

☐ False
False
False

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

n You need to write a simple loop that iterates over the integers from 1 to 10, and in each iteration, prints the sum of the current and the previous number. (In the first iteration, just take the previous number to be 0.)

For example:

$$1 + 0 = 1$$

$$2 + 1 = 3$$

$$3 + 2 = 5$$

...

$$10 + 9 = 19$$

Choose the program that will print the correct output (i.e.: only the sums).

1.0 point · Multiple choice · 4 alternatives

- ☐

```
i = 0
while i <= 10:
    print(i + (i-1))
    i += 1
```
- ☒

```
previous = 0
for i in range(1, 11):
    print(i + previous)
    previous = i
```

☐ Both of the above work

☐ Neither of above works

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

o Executing the following program will produce an error. Why?

```
def cube(num):  
    result = num ** 3  
    return f'The cube of {num} is: {result}.'  
  
print(result)
```

1.0 point · Multiple choice · 4 alternatives

- ☒ Result is a local name and cannot be referenced outside of the function.
- ☐ ** is not a valid Python operator.
- ☐ The syntax of the f-string is incorrect.
- ☐ You can only use either print or return, but not both.

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

p You already have a list named `list_1` with 2 lambda functions as follows:

```
list_1 = [lambda a, b: a + b, lambda a, b: a * b]
```

What would be printed by the following code snippet:

```
print(list_1[0](1, 2) ** list_1[1](1, 2))
```

1.0 point · Multiple choice · 4 alternatives

☐ 6

☐ 7

☐ 8

☒ 9

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

q You have the following function called main:

```
def main(x):  
    y = []  
    for i in x:  
        if len(i) == 0:  
            y.append(True)  
        else:  
            y.append(False)  
    return y[0] == y[1] == y[2]
```

What will be printed by the following lines:

```
print(main([{}, {}, {}]))  
print(main([[1], [2, 2], [3, 3, 3]]))  
print(main([[1], [2], [3]]))
```

1.0 point · Multiple choice · 4 alternatives

- ☐ True
☐ False
False
- ☐ False
☐ False
True
- ☐ True
☐ False
True
- ☒ True
☐ True
True

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

r How do you break a list into a list of lists, where each sublist contains 3 elements?

For example:

If you have the original list x as:

```
x = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
```

How do you create:

```
y = [['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i']]
```

1.0 point · Multiple choice · 4 alternatives

☒ `y = [x[i:(i + 3)] for i in range(0, 7, 3)]`

☐ `y = [x[i[0]::3] for i in enumerate(x)]`

☐ Both of the above work

☐ Neither of above works

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

s You have to write a function called `main` that accepts a dictionary as an input argument. Both the keys and the values of the input are integers. Return another dictionary that has the same keys as the input argument. For each output key, the corresponding value should be calculated as the sum of those values from the input dictionary whose keys are smaller than or equal to the output key in question.

For example:

```
main({5: 1, 2: 5, 4: 2})
```

should return a dictionary that is equal to:

```
{5: 8, 2: 5, 4: 7}
```

Which of the following programs works as intended?

1.0 point · Multiple choice · 4 alternatives

- ☐

```
def main(d):  
    result = {}  
    for output_key in d.keys():  
        result[output_key] = sum([v for k, v in d.items() if k <= output_key])  
    return result
```
- ☐

```
def main(d1):  
    running_total = 0  
    d2 = {}  
    for key in sorted(d1.keys()):  
        running_total += d1[key]  
        d2[key] = running_total  
    return d2
```
- ☒ Both of the above work as intended
- ☐ None of the above works as intended

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

t What is the result of the following program?

```
l1 = [1, 2, 3, 4]
l2 = [2, 3, 4, 5]

d1 = {}
for key, value in zip(l1, l2):
    d1[key] = value

d2 = {}
for key, value in enumerate(l2, 1):
    d2[key] = value

print(d1 == d2)
```

1.0 point · Multiple choice · 4 alternatives

- ☒ True
- ☐ False
- ☐ An error message.
- ☐ None of the above

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

u d1 = {1: 10, 2: 20, 3: 30, 4: 40}

The following codes all print the same value, except one. Which one?

1.0 point · Multiple choice · 4 alternatives

- ☒

```
result = 0
for key, value in d1:
    result += value
print(result)
```
- ☐

```
result = 0
for key in d1.keys():
    result += d1[key]
print(result)
```
- ☐

```
result = 0
for value in d1.values():
    result += value
print(result)
```
- ☐

```
result = sum(d1.values())
print(result)
```

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

v You have the following string

```
x = 'Amsterdam'
```

Which script, using the count string method, would print the following value:

1

1.0 point · Multiple choice · 4 alternatives

- ☐ `print(count(x, 'a'))`
- ☒ `print(x.count('a'))`
- ☐ Both scripts would deliver that result
- ☐ Neither script would deliver that result

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

w Take a look at the following function definition.

```
def calculation(func, *args):  
    result = 0  
    for el in args:  
        result += func(el)  
    return result
```

Which of the following function calls would return the number 30 as a result?

1.0 point · Multiple choice · 4 alternatives

- ☐ `calculation(lambda x: x + 5, 1, 2, 3, 4)`
- ☐ `calculation(lambda x: x**2, 1, 2, 3, 4)`
- ☒ Both function calls would lead to that result
- ☐ Neither function call would lead to that result

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly

x If you know the radius r of a circle and you want to calculate its area, you need the value of π from the math package and apply the formula:

$$\pi r^2$$

Suppose that the variable r already contains the radius of the circle. Which script does not print the correct answer for the area?

1.0 point · Multiple choice · 4 alternatives

- ☐ `import math`
`print(math.pi * r ** 2)`
- ☐ `from math import pi`
`print(pi * r ** 2)`
- ☒ `import math as constants`
`print(math.pi * r ** 2)`
- ☐ `from math import pi as constant`
`print(constant * r ** 2)`

Feedback

Feedback when the question is answered correctly

Feedback when the question is answered partially correctly

Feedback when the question is answered incorrectly